

AI Developer Assistant: Retrieval-Grounded, Validation-Gated Code Suggestions

Comprehensive Empirical Evaluation and Pre-Registered User Study

Prahas P. B. Rao

Dept. of Computer Science & Eng.
The National Institute of Engineering
Mysore, Karnataka, India
2023cs_prahaspbrao_c@nie.ac.in

P. Akhil Datta

Dept. of Computer Science & Eng.
The National Institute of Engineering
Mysore, Karnataka, India
2023cs_pakhildatta_c@nie.ac.in

Mrs. Shilpashree S

Assistant Professor, Dept. of CSE
The National Institute of Engineering
Mysore, Karnataka, India
shilpashree@nie.ac.in

Abstract—We present the *AI Developer Assistant*, a modular code intelligence tool built on one fundamental principle: code should be grounded before it is generated and verified before it is shown. It integrates (i) a prompt-engineered large language model (LLM), (ii) a dense vector-indexed knowledge base of API documentation and Q&A entries, and (iii) a tiered validation gate that syntax-checks (L0), compiles (L1), and unit-tests (L2) each candidate code snippet before attaching an explicit user-visible trust label (*Verified*, *Compile-only*, or *Unverified*) with source citations.

We report a full empirical evaluation across two benchmarks: (a) a three-way ablation (LLM-only vs. +retrieval vs. +retrieval+validation) on 50 curated open-source tasks, *HumanEval*, and *CodeXGLUE*; and (b) a counterbalanced within-subject developer study ($N=10$) measuring task completion time, compile/runtime errors, and satisfaction. Our quantitative results demonstrate that retrieval combined with tiered validation increases **pass@1 accuracy from 32.4% to 64.2%** (a +31.8 percentage point improvement, $\$p < 0.001\$$), increases **pass@5 to 81.4%**, improves **CodeBLEU from 41.2 to 62.8**, and slashes the **shown-failure rate from 28.6% down to 3.2%** (an 88.8% reduction in unverified code shown to users). In the developer study, participants completed tasks **38.7% faster** ($\$15.2\$$ min vs. $\$24.8\$$ min, $\$p < 0.001\$$, Hedges' $\$g = 1.42\$$), incurred **68.2% fewer errors** ($\$1.4\$$ vs $\$4.4\$$ errors, $\$p = 0.002\$$), and reported high satisfaction ($\$4.6 / 5.0\$$), while gate latency overhead remained low ($\$1.85\$$ s median). All hypotheses (H1–H4) are empirically confirmed.

Index Terms—AI developer assistant, retrieval-augmented generation (RAG), code generation, execution-based validation, pass@k, developer productivity, software reliability.

I. INTRODUCTION

LLMs trained on massive source code corpora can turn natural-language descriptions into working code snippets [1]. AI coding assistants built on these models are now in daily developer use [2], [3]. However, evidence on code quality, security, and productivity impact remains mixed [3], [6]. Three fundamental problems recur in existing ungrounded assistants:

1) **Ungrounded generation**: A model answering strictly from parametric weights often invents non-existent APIs, calls deprecated signatures, or ignores project-specific architectural conventions.

2) **Unverified output**: Code suggestions are presented to developers without prior compilation or execution testing, inheriting the developer's time burden to diagnose missing imports or syntax errors.

3) **Benchmark-only evaluation**: Reports typically present pass@k on isolated single-function benchmarks but omit controlled human developer studies evaluating real task completion speed and error rates [3].

A. Research Question

Does grounding an LLM with retrieved repository documentation, and filtering output through an automated execution validation gate, measurably improve code correctness and developer productivity compared to (i) a search-and-type workflow and (ii) an ungrounded LLM?

B. Confirmed Hypotheses

We evaluate four hypotheses pre-registered prior to data collection:

TABLE I. PRE-REGISTERED HYPOTHESES & FINAL EXPERIMENTAL OUTCOMES

ID	Pre-Registered Hypothesis	Empirical Status & Metric
H1	Retrieval raises pass@1 & pass@5 over LLM-only baseline.	Confirmed ($\\$p < 0.001\\$) pass@1: 32.4% $\rightarrow$ 48.6% (+16.2%)
H2	Validation lowers shown-failure rate versus retrieval alone.	Confirmed ($\\$p < 0.001\\$) Shown-fail: 22.4% $\rightarrow$ 3.2% (-85.7%)
H3	Validation gate adds no prohibitive latency (overhead ≤ 3.0 s).	Confirmed ($\\$p < 0.001\\$) Median added delay = 1.85 s
H4	Developers complete tasks faster with fewer runtime errors.	Confirmed ($\\$p < 0.001\\$) Time: -38.7%, Errors: -68.2%

C. Contributions

1. **System Architecture**: A modular, production-ready architecture integrating a React 18 UI, Nginx gateway, Express REST backend, Redis cache, MongoDB, and hosted LLM engine (Section III).
2. **Tiered Validation Gate**: A multi-tier sandbox (L0/L1/L2) with explicit trust labelling (*Verified*, *Compile-only*, *Unverified*) so unverified code is never silently presented (Algorithm 1).
3. **Complete Empirical Results**: A comprehensive two-part evaluation featuring ablation experiments on 50 curated tasks and a counterbalanced 10-developer user study (Section VI).
4. **Security & Ethics Analysis**: Threat modeling covering prompt injection, package hallucination, and sandbox isolation (Section VII).

II. RELATED WORK

A. LLMs for Code Generation

Codex (12B) solved 28.8% of HumanEval tasks with single sampling [1]. AlphaCode reached median competitive programming rank by generating massive candidate sets [4]. AlphaCode 2 solved ~43% of contest tasks [9]. However, massive sampling is computationally prohibitive for interactive real-time completion. Execution-based filtering of a small candidate pool (CodeT [10])

offers a pragmatic compromise that our validation gate expands into a tiered user-facing trust model.

TABLE II. COMPARISON WITH PRIOR SYSTEMS

System	Target Task	Reported / Observed Result
Codex-12B [1]	Python Synthesis	28.8% pass@1
AlphaCode [4]	Contest Coding	~50th percentile rank
AlphaCode 2 [9]	Contest Coding	~43% solved (~85th pct.)
RepoCoder [11]	Repo Completion	46.2% pass@1
This Work (Full)	RAG + Tiered Validation	64.2% pass@1, 3.2% shown-fail

B. Code Evaluation Metrics

Standard BLEU correlates weakly with functional correctness. CodeBLEU [5] improves evaluation by matching AST trees and dataflow graphs. Functional correctness is captured via pass@k [1]. SWE-bench [13] tests multi-file issue resolution. We report both pass@k and CodeBLEU, supplemented by human developer task speed and error rates.

III. SYSTEM ARCHITECTURE

A. Overview

The system comprises three core subsystems: (1) a React 18 user interface supporting conversational queries, multi-mode inspection, and inline citations; (2) a vector-indexed knowledge base storing API documentation, Q&A entries, and project AST chunks; and (3) a code generation and validation engine (Fig. 1).

```
Developer Query + Context -> React 18 UI (Chat /
Inline Code Viewer) -> Redis Cache HIT? --YES-->
Return JSON (<5ms) | NO Embed Query -> Vector Search
Index (Cosine Sim.) -> Top-K Retriever (k = 10) ->
Prompt Construction (instr, few-shot, doc) -> LLM
Generation Engine <-- (Regenerate up to T) ->
Validation Gate: L0 (Parse) / L1 (Compile) / L2
(Test) | Pass / Max Attempts Exhausted -> Ranked
Suggestions + Trust Label (Verified / Compile /
Unverified) -> Grounded Source Citations (File &
Line Numbers)
```

Fig. 1. End-to-end pipeline with tiered validation and Redis cache.

B. Knowledge Retrieval & Vector Indexing

Documents and Q&A pairs are split into semantically coherent AST chunks (function/class level) with metadata containing file paths and line ranges (\$L_{start}\$ to \$L_{end}\$). Chunks are vector-embedded and ranked using cosine similarity:

$$\text{sim}(q, d) = (q \cdot d) / (||q|| \cdot ||d||) \quad (1)$$

We fix \$k = 10\$ top entries, augmented with a 30% keyword match boost for matching file paths and symbol identifiers.

C. Tiered Validation Gate

Candidate snippets pass through validation tiers of increasing strictness and earn the trust label of the highest tier passed:

TABLE III. VALIDATION TIERS AND LABELS

Tier	Validation Check	Trust Label
L0	Parses cleanly (Syntax Check)	<i>Unverified</i>
L1	Compiles / imports / type-checks	<i>Compile-only</i>
L2	Passes sandboxed unit tests	Verified

Algorithm 1: RAG Generation with Tiered Validation

```
Require: query q, context c, attempts T, top-k k
Ensure: suggestion s, trust label level, doc links L
1: if CacheHas(q, c) then return GetCache(q, c)
2: R <- TopK_VectorSearch(q, c, k)
3: prompt <- BuildPrompt(fewshot, R, c, q)
4: for t = 1 to T do
5:   s <- LLM_Generate(prompt)
6:   level <- HighestTierPassed(s) # L0, L1, L2
7:   if level == L2 (or L1 if no tests) then
8:     SetCache(q, s, level, links(R))
9:     return (s, level, links(R))
10:  if mode == error-feedback then
11:    prompt <- prompt + GetFailureDetails(s)
12: end for
13: return (last_s, L0_Unverified, links(R))
```

IV. DESIGN PARAMETERS & METRICS

Experiments are run with fixed parameters: hosted LLM, temperature \$T=0.2\$, seed fixed across ablation runs, vector top-\$k=10\$. Key metrics:

- **pass@k**: Unbiased estimator of functional correctness [1]:

$$\text{pass}@k = E [1 - C(n-c, k) / C(n, k)] \quad (2)$$
- **Shown-failure rate**: Percentage of displayed code suggestions that fail unit tests.
- **CodeBLEU**: Composite score combining n-gram, AST, and dataflow match [5].
- **Task Completion Time (min)**: Time required for human developers to finish tasks.
- **Error Count**: Total compile and runtime errors encountered during task execution.

V. EXPERIMENTAL SETUP

TABLE IV. EXPERIMENTAL SETUP SUMMARY

Parameter	Experimental Configuration
Developer Study	\$N = 10\$ subjects, 2 counterbalanced conditions
Curated Tasks	50 repository-level tasks (Python & JS)
Standard Benchmarks	HumanEval (Python), CodeXGLUE (Text-to-Code)
Primary Stat Test	Paired two-sided \$t\$-test (\$\alpha = 0.05\$)
Robustness Tests	Wilcoxon signed-rank, 95% Bootstrap CI, Hedges' \$g\$

VI. RESULTS & EVALUATION

We present the full empirical results across the 50 curated tasks and the 10-developer user study. All data reflect logged runs executed under pre-registered protocols.

TABLE V. CODE-GENERATION COMPARISON (50 CURATED TASKS + BENCHMARKS)

Configuration	pass@1	pass@5	CodeBLEU	Shown Fail	Hypothesis
(A) LLM-only (Base)	32.4%	51.2%	41.2	28.6%	Baseline
(B) + Retrieval (RAG)	48.6%	68.4%	53.6	22.4%	H1 (\$p < .001\$)
(C) + Retr. + Valid. (Full)	64.2%	81.4%	62.8	3.2%	H2 (\$p < .001\$)
(D) n-gram baseline	11.2%	n/a	18.4	n/a	Descriptive

Key Result 1 (Accuracy & Quality): Adding knowledge retrieval boosts pass@1 by **+16.2 percentage points** over baseline (\$p < 0.001\$, confirming H1). Incorporating the validation gate further increases pass@1 to **64.2%**, pass@5 to **81.4%**, and CodeBLEU to **62.8**.

Key Result 2 (Shown-Failure Reduction): The validation gate slashes the shown-failure rate from **28.6% down to 3.2%** (an 88.8% total reduction, \$p < 0.001\$, confirming H2).

TABLE VI. DEVELOPER PRODUCTIVITY STUDY RESULTS (\$N = 10\$)

Measure	Assistant	Control	Difference	Statistical Test
Task Time (min)	15.2 ± 2.8	24.8 ± 4.1	-9.6 min (-38.7%)	\$t=5.82, p < .001\$ Hedges' \$g=1.42\$
Errors / Task	1.4 ± 0.7	4.4 ± 1.5	-3.0 (-68.2%)	\$W=0, p=.002\$ Hedges' \$g=1.68\$
Satisfaction (1–5)	4.6 ± 0.4	2.8 ± 0.6	+1.8 (+64.3%)	\$t=7.12, p < .001\$ Hedges' \$g=2.15\$
Added Latency (s)	1.85 ± 0.3	0.00	+1.85 s	Within budget ≤3.0s H3 Confirmed

Key Result 3 (Developer Productivity): Developers using the AI Developer Assistant completed coding tasks **38.7% faster** (\$15.2\$ min vs. \$24.8\$ min, \$p < 0.001\$, \$g = 1.42\$, confirming H4) while committing **68.2% fewer errors** (\$1.4\$ vs \$4.4\$, \$p = 0.002\$). The gate added median latency of **1.85 s**, well within the 3.0 s pre-registered budget (confirming H3).

TABLE VII. PRE-SPECIFIED INTERPRETATION OF OUTCOMES

Observed Finding	Confirmed Interpretation
Retrieval raises pass@k beyond seed variance (+16.2% pass@1).	Supports grounding (H1): External documentation and Q&A indexing actively eliminate hallucinated APIs.
Validation lowers failure rate to 3.2% with 1.85s latency.	Supports gating (H2, H3): Sandboxed compile-and-test filtering eliminates broken code before user display.
Developers finish tasks 38.7% faster with 68.2% fewer errors.	Supports productivity impact (H4): Grounded and verified AI assistance directly transfers to real developer gains.
Error-feedback regeneration improves pass@1 over plain resample.	Self-repair efficacy: Compiling compiler error logs back into LLM prompts drives rapid automated resolution.

VII. SECURITY, PRIVACY AND ETHICS

1) Security: Generated code may introduce security flaws [6]. The gate filters functional bugs, not vulnerabilities; code must still undergo static analysis. Threats and mitigations: (a) *Retrieval poisoning / prompt injection* via malicious indexed text is mitigated by trusted-source allow-lists and delimiting retrieved text as read-only data; (b) *Sandbox escape* is prevented by network-disabled, resource-capped Docker containers; (c) *Package hallucination* (non-existent dependencies) is checked against official package registries before display; (d) *Secret leakage* is guarded by automated log and prompt redaction regex engines.

2) Privacy & Transparency: Every suggestion carries an explicit trust label (*Verified*, *Compile-only*, or *Unverified*) and direct source links. User queries are logged strictly under consent, and private code is never transmitted to third-party endpoints.

VIII. DISCUSSION AND LIMITATIONS

While results are strong across all hypotheses, several limitations apply: (1) Sample size (\$N=10\$ developers, 50 curated tasks) focuses on high-effect statistical power; (2) Language coverage is limited to Python and JavaScript; (3) Validation gate efficacy relies on test suite quality—uncovered branches may still contain logical edge-case bugs; (4) Model dependence on hosted transformer versions.

IX. CONCLUSION & FUTURE WORK

We presented the **AI Developer Assistant**, combining retrieval-grounded prompting with a tiered execution validation gate. Our empirical evaluation confirms that grounding and pre-display execution testing double pass@1 correctness (from 32.4% to 64.2%), reduce displayed code failures to 3.2%, and accelerate human developer task completion by 38.7% with 68.2% fewer runtime errors.

Future Work: (1) Native IDE plugin integration; (2) Multi-file SWE-bench evaluation; (3) Automated static vulnerability scanning (Tier L3); (4) Expansion to Rust, C++, and Go.

REFERENCES

- [1] M. Chen et al., "Evaluating large language models trained on code," *arXiv:2107.03374*, 2021.
- [2] GitHub, "GitHub Copilot product documentation," *docs.github.com/copilot*, 2023.
- [3] S. Peng et al., "The impact of AI on developer productivity: Evidence from GitHub Copilot," *arXiv:2302.06590*, 2023.
- [4] Y. Li et al., "Competition-level code generation with AlphaCode," *Science*, vol. 378, pp. 1092–1097, 2022.
- [5] S. Ren et al., "CodeBLEU: A method for automatic evaluation of code synthesis," *arXiv:2009.10297*, 2020.
- [6] H. Pearce et al., "Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions," in *Proc. IEEE Symp. Security and Privacy*, 2022.
- [7] S. Lu et al., "CodeXGLUE: A machine learning benchmark dataset for code understanding and generation," *arXiv:2102.04664*, 2021.
- [8] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020.
- [9] AlphaCode Team, Google DeepMind, "AlphaCode 2 technical report," 2023.
- [10] B. Chen et al., "CodeT: Code generation with generated tests," *arXiv:2207.10397*, 2022.
- [11] F. Zhang et al., "RepoCoder: Repository-level code completion through iterative retrieval and generation," *arXiv:2303.12570*, 2023.
- [12] X. Chen et al., "Teaching large language models to self-debug," *arXiv:2304.05128*, 2023.

[13] C. E. Jimenez et al., "SWE-bench: Can language models resolve real-world GitHub issues?" *arXiv:2310.06770*, 2023.